Prose as a shell language

Status: Stage 1a and 1b; the script reader, the first shell effects, and the `with` connective are implemented. Scope decisions: script-only first, REPL later; a new prose shell that calls external commands, not a POSIX clone; unrestricted first, with the capability work named as debt. `scale --prose <path>` reads and runs a `.prose` script on the kernel stack (D149). Both spellings execute commands: `run("echo" "hello")` and `run "echo" with "hello".` (D183, D187). `run` and `capture` are host words: the Prelude holds no process code, and the `shell-host` component owns every OS effect.

What already fits

- The concatenative stack is a pipeline model: a command produces a value or a stream and the next command consumes it.
- The prose surface already has lowercase words, spaced vocabulary names, `of` phrases, text with spaces, calls, matches, and a sentence sequence.
- The ABI already has the effect seam: `scale_host_call` and `scale_transport_send`. The shell host is the first real consumer.
- The actor and channel model can carry jobs and streams later.
- Failures are values, so an exit status is a value, not an exception.
- The CLI is NativeAOT, so script startup is native-fast.

Decisions

1. **Script-only first.** A script is a `.prose` file run by a new CLI mode; there is no interactive REPL in Stage 1.
2. **A new prose shell.** It calls external commands and does not reimplement a POSIX grammar. The surface is prose, not `/bin/sh`.
3. **Unrestricted first.** Stage 1 runs with the user's authority and no capability sandbox. This is deliberate debt, closed before any untrusted script is run.
4. **Effects at the edge.** The kernel and the Prelude gain no process, file, environment, or signal words. A `shell-host` component owns them and is reached through the existing host-call seam.

The command model

Stage 1 executes one command sentence at a time. The surface is quote-free:

```
run echo with hello.
run cat with file.txt.
capture echo with hello.
```

A bare word after a shell verb is a text argument. The older quoted call spelling remains accepted for compatibility, but it is not the target surface. Both spellings lower to the same host-word call, so the surface has one semantics (D183, D187).

`run` starts one external command, forwards the child's standard output and standard error to the script's streams, and pushes the exit status as a number. `capture` forwards the child's standard error and pushes the child's standard output as text. Both record the status as the script's last command status. Arguments are separate text values. A non-text argument is a named refusal.

Stage 2 adds streams and composition with stack-connected command sentences:

```
capture echo with hello.
feed cat.
```

`feed` consumes the text on the stack, sends it to the next program's standard input, and pushes the result. File redirection uses `save`:

```
capture echo with hello.
save out.txt.
```

`save` consumes the text on the stack and writes it to the named file. Both are shell-effect host words and require the shell capability. No pipe or redirection symbol is added.

Stage 3 adds variables, environment, and the working directory:

```
let name = "src".
run ls with name.
change directory to "src".
```

Stage 4 adds the REPL and job control. Stage 5 adds capabilities, cross-platform polish, and release AOT tests.

Effects and the ABI

The shell words are **host words**, not Prelude primitives. `EvaluationState` carries an optional `HostWordAdapter`. When the dispatcher does not handle a token, it offers the token to the adapter before reporting an unknown word. Lowering maps a shell call to a host-word token that carries its argument count, so a variadic command cannot consume values that were already on the stack. A host token with no attached adapter is a named refusal, never a silent success.

A new `components/shell-host` implements the adapter:

- start a process with a program path and an explicit argument list;
- forward stdout and stderr for `run`, and capture stdout for `capture`;
- return the exit status;
- later: pipes, file redirection, environment, working directory, signals.

The kernel stays pure, and the Prelude contains no process, file, environment, or signal code. `shell-host` is the only component with OS authority in this slice; the base attaches it when it runs a `.prose` file. The ABI's capability registry is the place to enforce permissions in Stage 5.

Stage 1 uses `System.Diagnostics.Process` with `UseShellExecute = false`, an explicit argument list, and redirected standard output and error. There is no shell interpolation, globbing, or metacharacter handling; an argument is the text it is. The child inherits the script's standard input, environment, and working directory.

Script semantics

- A `.prose` script is a sentence sequence, run in order on one stack.
- `run` pushes the exit status as a number; `capture` pushes the captured standard output as text.
- The script's exit code is the last command status (from `run` or `capture`), or zero when the script contains no command.
- `run` forwards the child's standard output and standard error; `capture` forwards standard error and returns standard output. The streams are the script's streams.
- The environment, working directory, and standard input are inherited from the host.
- A non-zero status is a value, not an immediate abort. `and then`, `or else`, and `set -e`-style behavior are later surface.
- A parse or lowering refusal, a missing program, or a process that cannot start is a named refusal with a non-zero script status, never a silent success. A missing program names the program text and returns status 1.

Testing

Stage 1a and 1b acceptance, implemented in the suite:

1. `capture("echo" "hello")` returns `"hello\n"`.
2. `run("false")` pushes status 1 and the script exits 1.
3. `run "echo" with "hello".` parses to the same call as `run("echo" "hello")`.
4. `capture "echo" with "hello".` returns `"hello\n"`.
5. an argument with spaces arrives as one argument;
6. a missing program is a named refusal;
7. the environment and working directory are inherited;
8. no shell metacharacter is interpreted;
9. an unattached host refuses a shell word;
10. the AOT binary runs the same script.

The `with` fixtures are parse and equivalence tests: the target spelling and the call spelling produce the same tree and the same host-word tokens.

The slice is tested at three levels: lowering (the call becomes a host-word token carrying its arity), the adapter (a fake process records the program and arguments and checks the stack result), and the process host (`/bin/echo`, `/bin/false`, a missing program, a spaced argument, an inherited environment and working directory). The product smoke is the release-time AOT check.

Security

Unrestricted first is explicit, not accidental. The default `ProcessShell` is unrestricted; a host that accepts untrusted scripts constructs a restricted one instead.

Policy field	Effect
allowed programs	the program string must be in the allowed set
write roots	`save` must resolve inside one allowed root
environment	restricted policy may clear the inherited environment

The policy is enforced at the effect boundary, not in the Prelude. Read paths, network, and package signatures remain later security work; the shell policy is the first enforced subset. A denied program or write path is a named refusal, never a silent fallback.

Staging

1. **Stage 1a.** Script runner (landed, D149), the host-word adapter, `run`, `capture`, last-command status, inherited environment and working directory (landed, D183).
2. **Stage 1b.** The `with` connective: `run "echo" with "hello".` and `capture "echo" with "hello".` (landed, D187).
3. **Stage 2.** Streams, pipelines, file redirection.
4. **Stage 3.** Variables, environment, working directory, the `let`/`if` surface.
5. **Stage 4.** REPL, background jobs, signals.
6. **Stage 5.** Capability sandbox, cross-platform, AOT release tests.

What this does not claim

This slice does not add pipelines, file redirection, variables, an interactive REPL, or a capability sandbox. It adds the host-word seam, the first two effects, and the `with` connective. `--prose-typecheck` does not infer host words yet; run a shell script with `--prose`.